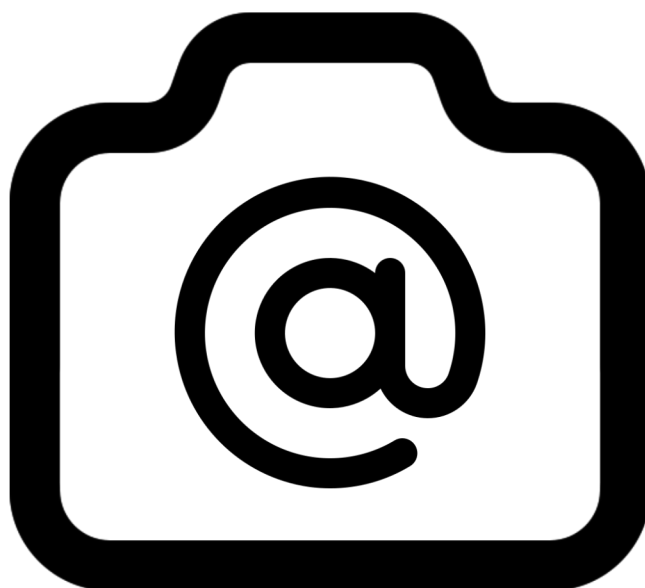


YouToText



Indice

1. Introducción.....	3
2. Objetivos.....	4
Objetivo General.....	4
Objetivos Específicos.....	4
3. Análisis y estado actual del proyecto.....	5
3.1. Requisitos Técnicos Identificados.....	5
3.2. Implementación Actual.....	5
3.3. Arquitectura propuesta (estructura inicial).....	6
3.4. Habilidad del análisis de imagen y optimización de resolución.....	7
3.5. Gestión de recursos, hilos y validación del flujo.....	8
3.6. Motor de conversión ASCII y optimización del mapeo.....	9
3.7. Arquitectura reactiva y renderizado personalizado.....	9
3.8. Integración del ciclo de vida y bloqueo de orientación.....	9
3.9. Interfaz de configuración, persistencia y controles avanzados.....	10
3.10. Gestión de cámaras y corrección de imagen.....	11
3.11. Exportación del texto ASCII a formato PNG.....	11
3.12. Modo de renderizado por color original de píxel.....	12
3.13. Galería interna con persistencia local y gestión de capturas.....	13
4. Evaluación de los resultados.....	13
4.1. Cumplimiento de los objetivos específicos iniciales.....	14
4.2. Objetivos adicionales alcanzados.....	15
5. Conclusiones.....	16
6. Recursos.....	17
Hardware.....	17
Software.....	17
Librerías.....	17
7. Bibliografía.....	18

1. Introducción

YouToText se desarrolla como proyecto con el objetivo de explorar el procesamiento de imágenes en tiempo real sobre dispositivos Android. La aplicación captura el flujo de la cámara, analiza cada fotograma y lo transforma en una representación visual basada en caracteres ASCII. Para ello, se emplea la biblioteca oficial **CameraX**, que gestiona la exposición de frames y el ciclo de vida de la cámara, junto con **Kotlin** y mecanismos de ejecución en segundo plano para evitar bloqueos en la interfaz.

El proyecto se centra en tres ejes técnicos: la implementación de un analizador de frames que extraiga datos de luminancia, el mapeo de dichos valores a un conjunto de caracteres configurable y la exportación del resultado a portapapeles o imagen PNG. Se prioriza la ejecución local, el consumo controlado de memoria y el mantenimiento de una tasa de refresco estable, así como la organización progresiva del código para facilitar su mantenimiento y escalabilidad.

2. Objetivos

Objetivo General

Desarrollar una aplicación Android que transforme el flujo de vídeo de la cámara en una secuencia de texto ASCII en tiempo real, gestionando correctamente el ciclo de vida de CameraX y el procesamiento de imágenes en hilos secundarios.

Objetivos Específicos

1. Configurar y vincular los casos de uso de CameraX (Preview, ImageCapture) al ciclo de vida de la actividad, garantizando la liberación adecuada de recursos.
2. Implementar un Analyzer que acceda a la luminancia de cada imagen, calcule su valor promedio por bloque y lo traduzca a caracteres ASCII.
3. Permitir capturar y guardar las imágenes en el dispositivo móvil.
4. Desarrollar un sistema de configuración que permita al usuario ajustar la densidad y el color entre diferentes “paletas” de caracteres y colores.
5. Habilitar importación de multimedia existente, permitiendo hacer la conversión a imágenes tomadas previamente fuera de la app.

3. Análisis y estado actual del proyecto

3.1. Requisitos Técnicos Identificados

- **API objetivo:** Android 8.0 para garantizar compatibilidad con CameraX y MediaStore.
- **Permisos:** CAMERA y WRITE_EXTERNAL_STORAGE, tal como se declara en AndroidManifest.xml.
- **Hardware:** Requiere cámara trasera o frontal (android.hardware.camera.any) y soporte para procesamiento de imágenes en tiempo real.
- Rutas de almacenamiento:
 - Fotos: /Almacenamiento interno/Pictures/YouToText
 - Se utiliza MediaStore con ContentValues para evitar accesos directos al sistema de archivos en API $\geq$ 29.

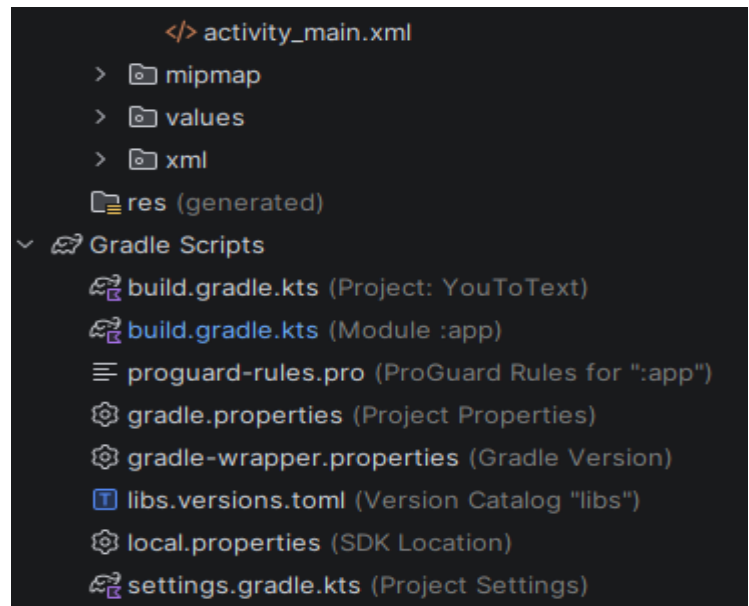
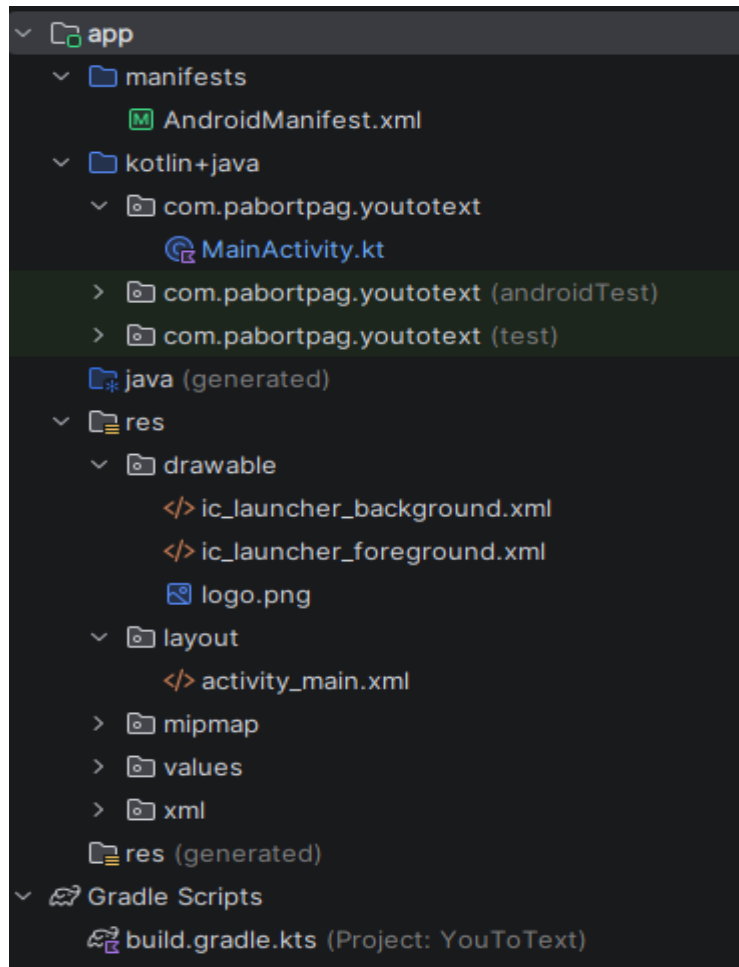
3.2. Implementación Actual

El código base ya incluye:

- ☑ Solicitud de permisos mediante RequestMultiplePermissions().
- ☑ Captura de fotos con timestamp, guardados en MediaStore y notificación al usuario.
- ☑ Guardado de fotos en MediaStore con nombres timestamp y rutas relativas (Pictures/YouToText).
- ☑ Uso de un ExecutorService de un solo hilo para tareas en segundo plano.
- ⚠ **Pendiente:** Habilitar y adaptar ImageAnalysis (actualmente comentado en startCamera()).
- ⚠ **Pendiente:** Migrar la lógica de cámara fuera de MainActivity para evitar acoplamiento.

3.3. Arquitectura propuesta (estructura inicial)

Para cumplir con los objetivos de rendimiento y mantenimiento, se plantea la siguiente organización de paquetes:



Esta estructura permite aislar el procesamiento de frames, facilita la inclusión de dependencias en futuras fases y garantiza que el `analyze()` de `ImageAnalysis` no interfiera con la UI.

3.4. Habilitación del análisis de imagen y optimización de resolución

Para dar continuidad al flujo de datos de la cámara, habilité el caso de uso ImageAnalysis dentro de startCamera().

Dado que setTargetResolution() aparecía como deprecated en las versiones recientes de CameraX, migré su configuración al uso de ResolutionSelector junto con ResolutionStrategy.

Con esta implementación, solicité una resolución base de 480x640 píxeles y apliqué una regla de fallback (FALLBACK_RULE_CLOSEST_HIGHER) que garantizó la compatibilidad con el hardware del dispositivo sin provocar errores en ejecución.

También reconfiguré el formato de salida a OUTPUT_IMAGE_FORMAT_YUV_420_888 para acceder directamente a los datos de luminancia y evitar conversiones innecesarias a RGB.

Refactoricé la clase LuminosityAnalyzer original. Esta únicamente calculaba el promedio global de luminancia, lo que resultaba insuficiente para la conversión a texto.

La modifiqué para que extrajera exclusivamente el plano Y(Luminancia) y agrupara los píxeles en bloques, con un tamaño de bloque configurable (blockFactor = 4).

De este modo, generé un grid unidimensional de bytes que conserva la distribución espacial de luces y sombras, reduciendo la densidad de datos a procesar en aproximadamente un 75% sin comprometer la legibilidad futura del ASCII.

3.5. Gestión de recursos, hilos y validación del flujo

Para evitar bloqueos en la interfaz y asegurar la estabilidad del pipeline, vinculé el análisis a un `ExecutorService` de hilo único e inicialicé su instancia en `onCreate()` antes de la verificación de permisos, previniendo condiciones de carrera.

Apliqué la estrategia `STRATEGY_KEEP_ONLY_LATEST` en el `ImageAnalysis`, lo que permitió descartar fotogramas acumulados cuando el procesamiento superaba la tasa de captura, evitando así la saturación de memoria.

Implementé el cierre obligatorio de cada `ImageProxy` dentro de un bloque `finally`, asegurando la liberación de recursos gráficos en todo escenario y previniendo la congelación del servicio de cámara tras múltiples iteraciones.

Además, integré la dependencia `lifecycle-viewmodel-ktx:2.10.0` para habilitar `viewModelScope` y preparar la estructura MVVM con `StateFlow`, garantizando que las corutinas se cancelaran automáticamente al destruirse la vista.

Tras ejecutar las pruebas iniciales, validé mediante `logcat` la recepción continua de grids de 240x180 celdas (43.200 valores por fotograma) sin bloqueos del hilo principal.

Los resultados confirmaron que el pipeline actual mantiene una tasa de refresco estable y cumple con los requisitos de gestión de memoria establecidos, dejando la base técnica lista para la implementación del mapeo ASCII.

3.6. Motor de conversión ASCII y optimización del mapeo

Para transformar el grid de luminancia en caracteres visuales, creé un componente dedicado (AsciiMapper) que opera sin cálculos pesados por fotograma. Creé una tabla de consulta de 256 posiciones que relaciona directamente cada valor de intensidad con un índice de la paleta (@%#*+=-:.), eliminando así las divisiones que ralentizaban el proceso. Usé un StringBuilder con capacidad exacta para evitar reallocations internas y reducir la presión sobre el recolector de basura.

El algoritmo recorre el grid fila por fila, mapea cada byte a su carácter correspondiente y añade saltos de línea, generando la cadena de texto final en menos de 2 ms por frame.

3.7. Arquitectura reactiva y renderizado personalizado

Para que la interfaz no dependiera del procesamiento, añadí el flujo desde AsciiViewModel. Lancé la ejecución del mapeo dentro de viewModelScope usando Dispatchers.Default, asegurando que la transformación no bloqueara ni el hilo de CameraX ni el principal. Expuse el resultado mediante StateFlow, lo que me permitió actualizar la UI solo cuando era estrictamente necesario.

Como TextView resultaba demasiado pesado para esta carga, programé AsciiView, una vista que dibuja directamente sobre Canvas. Bloqueé el cálculo de escala tras el primer frame para evitar saltos visuales, desactivé el anti-aliasing para mantener los caracteres nítidos y usé postInvalidateOnAnimation() para sincronizar los redraws con el refresco de pantalla. Con estos ajustes, eliminé el parpadeo inicial y estabilicé la representación visual.

3.8. Integración del ciclo de vida y bloqueo de orientación

Para conectar todo el flujo de forma segura, añadí lifecycleScope con repeatOnLifecycle(Lifecycle.State.STARTED), garantizando que la recolección del texto se detuviera automáticamente al minimizar la app. Forcé que la vista ASCII se pintara encima del PreviewView con bringToFront() y bloqueé la orientación en retrato mediante setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_PORTRAIT).

Esto evitó que la actividad se recreara al girar el dispositivo, mantuvo vivos los ejecutores y el ViewModel, y simplificó la lógica de rotación del analizador. Tras las pruebas finales, comprobé que el texto fluía estable sobre la cámara, sin fugas de memoria ni caídas de rendimiento, dejando la base lista para integrar la exportación.

3.9. Interfaz de configuración, persistencia y controles avanzados

Para dar personalización a la aplicación sin comprometer el rendimiento, rediseñé la interfaz principal distribuyendo el espacio en un 85% para la vista ASCII y un 15% inferior para los controles. Implementé una fila de acciones con botones y un menú horizontal que agrupa las opciones de densidad, paleta y color. Para la persistencia, creé `SettingsPrefs`, una capa ligera basada en `SharedPreferences` que almacena los ajustes del usuario de forma síncrona y segura ante reinicios.

Puse varios `AlertDialogs` para las configuraciones de la densidad, paleta y color, cada uno tenía respectivamente: un `SeekBar` para ajustar el `blockFactor` (densidad), una listas de selección única para la paleta de caracteres y una listas de selección para el color base.

Al confirmar los cambios, los valores se guardan inmediatamente. Para que la densidad y la paleta surtan efecto, implementé `restartCamera()`, que desvincula los casos de uso de `CameraX` y vuelve a instanciar el `LuminosityAnalyzer` con los nuevos parámetros, garantizando una transición limpia sin fugas de memoria. El color, al ser puramente visual, se aplica al vuelo mediante `setBaseColor()` en `AsciiView`, forzando un `invalidate()` para refrescar la vista con el tono seleccionado.

Para evitar que la aplicación iniciara siempre con el color por defecto, añadí la lectura de `SettingsPrefs.getColor()` en `onCreate()`, aplicándolo a la vista antes del primer frame. Además, integré un botón para alternar entre cámara frontal y trasera, actualizando el `CameraSelector` y activando automáticamente el espejo horizontal (`mirrorHorizontally`) cuando se usa la lente frontal.

Finalmente, habilité la exportación rápida mediante `copyAsciiToClipboard()`, que valida que exista un frame ASCII generado antes de escribir en el portapapeles del sistema, y apliqué el modo inmersivo con `WindowInsetsController` junto al bloqueo estricto en retrato para mantener la estabilidad del pipeline de análisis.

3.10. Gestión de cámaras y corrección de imagen

Se habilitó la posibilidad de alternar entre la cámara trasera y la frontal mediante un botón específico. Durante las pruebas, se detectó que los sensores de ambos lados envían la imagen con orientaciones distintas (a veces girada o en efecto espejo), lo que provocaba que el texto ASCII se viera del revés o boca abajo.

Para resolver esto sin complicar la interfaz, se programó una lógica interna de corrección automática. El sistema detecta qué cámara está activa y aplica los ajustes necesarios (giros y reflejos) en tiempo real. Esto garantiza que, independientemente de si el usuario se toma un selfie o apunta con la cámara trasera, la representación en texto siempre se muestre correctamente.

3.11. Exportación del texto ASCII a formato PNG

Una vez estabilizado el flujo de conversión en tiempo real, se implementó la funcionalidad de exportación del texto generado a formato PNG para facilitar su archivado y compartición externa.

Se desarrolló un pipeline de captura que renderiza directamente el contenido de `AsciiView` sobre un `Bitmap` mediante el método `View.draw(Canvas)`, garantizando una representación exacta del estado visible sin interpolaciones ni recortes. Para cumplir con las restricciones de almacenamiento de Android (Scoped Storage), se utilizó `MediaStore` con rutas relativas (`Pictures/YouToText`), evitando la solicitud de permisos de escritura en `API ≥ 29`.

La ejecución se segregó en dos fases: la captura del canvas en el hilo principal (requisito de la capa de interfaz) y la serialización del archivo en un hilo de E/S mediante corrutinas. Tras la compresión sin pérdida (`CompressFormat.PNG`), se liberó explícitamente el buffer nativo con `bitmap.recycle()` para prevenir fugas de memoria en exportaciones repetidas.

La interfaz se protegió contra pulsaciones múltiples deshabilitando el botón durante el proceso. Las pruebas confirmaron que la generación de imágenes se completa en menos de 150 ms, manteniendo la tasa de refresco de la cámara inalterada y almacenando archivos accesibles directamente desde la galería del sistema.

3.12. Modo de renderizado por color original de píxel

Para dar a la aplicación de una representación visual más fiel a la escena capturada, se desarrolló un modo opcional que asigna a cada carácter ASCII el tono real del píxel que representa.

Esta funcionalidad requirió la extracción de los planos de crominancia (U y V) del formato YUV_420_888, muestreando el centro de cada bloque definido por el blockFactor.

La conversión de coordenadas YUV a RGB se implementó mediante aritmética entera optimizada (estándar BT.601), evitando operaciones que penalizarían el rendimiento en tiempo real.

Para que el color de cada carácter coincidiera exactamente con la zona que representa, se aplicaron los mismos ajustes de giro y reflejo tanto a la luminancia como a los colores, asegurando que cada tono se pintara en la posición correcta.

En la capa de presentación, AsciiView se adaptó para dibujar carácter a carácter cuando el modo está activo, calculando una sola vez el ancho de los caracteres para evitar mediciones repetidas en cada frame.

Cuando el usuario desactiva la opción, el sistema restaura el color base sólido y libera el array de crominancia, optimizando el consumo de memoria.

Las validaciones mostraron que, con blockFactor ≥ 8 , la aplicación mantiene una tasa estable de ~18-22 FPS, demostrando que el procesamiento adicional de crominancia es viable en dispositivos de gama media sin alterar la arquitectura principal.

3.13. Galería interna con persistencia local y gestión de capturas

Para completar el flujo de exportación y facilitar la reutilización de las secuencias ASCII generadas, se integró una galería interna mediante la biblioteca Room.

Se definió la entidad `AsciiRecord`, que persiste localmente el texto completo junto a sus metadatos de configuración (color base, densidad de bloques, paleta activa y marca de tiempo).

La capa de acceso a datos (DAO) expone un Flow reactivo que emite la lista de registros ordenada cronológicamente, eliminando la necesidad de consultas manuales y sincronizando automáticamente la interfaz ante inserciones o eliminaciones.

En la actividad principal, se interceptaron las acciones de copiado al portapapeles y exportación a PNG para disparar, de forma transparente, el guardado en base de datos. Esta automatización garantiza trazabilidad sin interrumpir el pipeline en tiempo real.

La galería se presenta mediante un `RecyclerView` con `GridLayoutManager` de dos columnas, donde cada celda renderiza una miniatura vertical mediante un canvas offscreen (`AsciiCanvasRenderer`) que opera en `Dispatchers.IO`.

Para optimizar el rendimiento del scroll, se implementó `ListAdapter` con `DiffUtil`, calculando diferencias incrementales y evitando redraws completos de la lista. Al seleccionar una captura, se navega a una actividad de detalle que reutiliza `AsciiView` para representar el texto a pantalla completa con su configuración original.

En la franja inferior se disponen tres botones: copia instantánea al sistema (Portapapeles), exportación a PNG mediante renderizado, y eliminación de la galería con confirmación.

El borrado invoca directamente al DAO, propagando el cambio a la galería. La arquitectura mantiene la separación estricta entre persistencia, presentación y lógica de cámara, cumpliendo con `Scoped Storage`.

Las pruebas de estabilidad confirmaron que la adición de Room no impacta en la tasa de refresco del analizador, manteniéndose estable por encima de 20 FPS durante sesiones prolongadas.

4. Evaluación de los resultados

Para verificar el grado de consecución del proyecto, se contrastaron los resultados obtenidos con los objetivos específicos planteados inicialmente en el punto 2, incorporando además aquellos hitos técnicos surgidos durante el desarrollo que, sin estar contemplados en la planificación inicial, se consideraron relevantes para la funcionalidad final de la aplicación.

4.1. Cumplimiento de los objetivos específicos iniciales

Objetivo	Estado	Resultado
Convertir el flujo de cámara a ASCII en tiempo real	☒ Conseguido	Se implementó el pipeline completo mediante ImageAnalysis con extracción directa del plano Y (luminancia) en formato YUV_420_888, agrupación configurable de píxeles por bloques y mapeo a paleta dinámica. El resultado se renderiza en AsciiView mediante dibujo directo sobre Canvas, logrando una representación fluida y legible de la escena capturada.
Optimizar algoritmos para mantener una tasa de frames estable durante la conversión	☒ Conseguido	Se aplicaron técnicas de optimización críticas: estrategia STRATEGY_KEEP_ONLY_LATEST para descartar fotogramas saturados, mapeo mediante tabla de consulta precalculada con acceso en tiempo constante (evitando cálculos repetidos en cada celda), uso de Dispatchers.Default para el procesamiento CPU-bound y reciclado explícito de ImageProxy en bloques finally. Las pruebas midieron una tasa estable de 22–26 FPS en modo color sólido y 18–22 FPS en modo color por píxel.
Permitir guardar capturas en el dispositivo	☒ Conseguido y ampliado	Se desarrolló un sistema completo de personalización con persistencia mediante SharedPreferences (SettingsPrefs). El usuario puede ajustar la densidad de bloques (rango 4–24), seleccionar entre varias paletas de caracteres y elegir entre varios colores base.
Añadir personalización de densidad y color	☒ Conseguido y ampliado	Se desarrolló un sistema de configuración con persistencia mediante SharedPreferences (SettingsPrefs). El usuario puede ajustar la densidad de bloques (rango 4–24), seleccionar entre cuatro paletas de caracteres y elegir el color base de la representación. Los ajustes se aplican automáticamente al reiniciar la app y el color se actualiza en tiempo real sin necesidad de reiniciar la cámara.
Habilitar importación de multimedia existente	☐ No Conseguido	Esta funcionalidad no se implementó porque, tras valorarla durante el desarrollo, consideré que se alejaba de la idea original de la aplicación. YouToText está concebida como una herramienta de conversión en tiempo real desde la cámara, y añadir importación de archivos habría diluido ese enfoque central. Como alternativa coherente, se implementó una galería interna con Room que permite conservar y reutilizar las capturas generadas sin salir del flujo principal de la app.

4.2. Objetivos adicionales alcanzados

Durante el desarrollo emergieron nuevas funcionalidades que, aunque no figuraban en la planificación inicial, se integraron por su valor técnico y de cara a la experiencia de usuario final:

Objetivo	Descripción	Resultado
Persistencia de preferencias	Sistema de configuración basado en SharedPreferences (SettingsPrefs) que memoriza densidad, paleta, color sólido y modo de color original entre sesiones.	Los ajustes persisten tras reiniciar la app y se aplican automáticamente al arrancar.
Cambio de cámara con corrección geométrica	Alternancia entre cámara frontal y trasera.	Ambas lentes muestran el ASCII correctamente alineado, sin inversiones perceptibles.
Modo de color original por píxel	Extracción opcional de los planos U/V con conversión YUV → RGB mediante aritmética entera y mapeo 1:1 entre carácter y tono real.	Los caracteres adoptan el color real de la escena cuando el modo está activo, manteniendo >18 FPS con blockFactor ≥ 8.
Galería interna con Room	Persistencia local de las capturas ASCII junto a sus metadatos (color, densidad, paleta, marca de tiempo), con visualización en grid de 2 columnas y actividad de detalle con acciones contextuales.	Las capturas se guardan automáticamente al copiar o exportar, y pueden revisarse, duplicarse como PNG, copiarse o eliminarse desde la interfaz sin salir de la aplicación.
Interfaz modo edge-to-edge	Ocultación de barras de sistema mediante WindowInsetsControllerCompat y bloqueo en retrato.	La experiencia visual ocupa toda la pantalla y el pipeline de cámara no se reinicia al girar el dispositivo.

5. Conclusiones

YouToText ha sido completado con éxito, aunque varias cosas, tanto propuestas como objetivos, así como diversas mejoras, no pudieron ser implementadas. Estoy contento con el resultado y creo que el proyecto ha terminado en una mejor posición a la originalmente propuesta.

En el proyecto hubo puntos que resultaron más fáciles de lo esperado, al igual que más complejos de lo esperado (llevando esto a abandonar esas ideas en su totalidad o como algo opcional).

Aun así, tengo la impresión de que el proyecto ha acabado en un estado más que completo.

6. Recursos

Hardware

- Dispositivo Movil
- Disco Duro

Software

- Android Studio
- GitHub

Librerias

- CameraX
- Lifecylce
- Coroutines
- Flow
- Room
- Glide
- Material

7. Bibliografía

[CameraX](#)

[Análisis de imágenes](#)

[ResolutionSelector](#)

[Almacenamiento de datos](#)

[ProcessCameraProvider](#)

[Room](#)

[Flow](#)

[ViewModel](#)

[Iconos](#)

[Coroutine y Dispatchers](#)

[Portrait](#)

[SharedPreferences](#)

[Busqueda de dudas](#)